

Instructions: Attempt all the questions. Write cleanly and clearly.

- (10 points) Given a sequence $A_1, A_2, \dots, A_n$ of integers, we are interested in finding i and j with $1 \leq i \leq j \leq n$, that maximizes $\sum_{k=i}^j A_k$. Design a divide and conquer algorithm which solves this problem in $O(n \log n)$. Write down the recurrence relation for its running time and its solution.
- (4+4+2 points) Given a graph $G(V, E)$, a subset $S \subseteq V$, is called an independent set, if there are no edges between the vertices in S . Here, we consider the problem of finding the largest independent set in a rooted tree.

Write down the recursive relation, and briefly describe (70 words) how you will solve it via dynamic programming in linear time. Also, briefly analyse (30 words) its running time.

- (4+1 points) We are given an undirected, unweighted graph $G = (V, E)$ and we would like to find a cut $C(S, V \setminus S)$ with value as small as possible. The weight of a cut $w(C)$ is simply the number of edges with one endpoint in S and the other endpoint in $V \setminus S$.
 - Design a deterministic algorithm to solve this problem using the Ford-Fulkerson Algorithm.
 - What is the time complexity of your algorithm?
- (3x5 =15 points) We define a *most vital arc* of a flow network as an arc (edge) whose deletion causes the largest decrease in the maximum $s - t$ flow value. Let f be an arbitrary maximum $s - t$ flow. Either prove the following claims or show through counterexamples that they are false (**stating true or false without proof or counterexample will not award any score**):
 - A most vital arc is an arc e with the maximum value of $c(e)$.
 - A most vital arc is an arc e with the maximum value of $f(e)$.
 - A network might contain several most vital arcs.
- (6+2 points) Consider the following variation of Karger's algorithm (CONTRACT). Let $l : \mathbb{N} \rightarrow \mathbb{N}$ be a monotone function such that $1 \leq l(n) < n$ for every $n \in \mathbb{N}$. Basically, this algorithm contracts the graph till it reaches a reasonably small size of $l(n)$ vertices, where $|V| = n$, and then solve the min-cut problem for this smaller graph using any efficient deterministic algorithm.

DETRAN(1)

Input: A Multigraph $G=(V,E)$ of n vertices, where n is a natural number ≥ 3 .

STEP 1: Perform the algorithm CONTRACT(G) in order to get a multigraph G/F of $l(n)$ vertices.

STEP 2: Apply the best known deterministic algorithm on G/F to compute an optimal cut D of G/F .

Output: D

- Prove that algorithm DETRAN(l) finds an optimal solution with probability at least $\frac{\binom{l(n)}{2}}{\binom{n}{2}}$.
 - Is DETRAN(l) guaranteed to return always the correct answer? Justify your answer.
- (7 points) Suppose you have an unsorted list with n elements, where you are searching for an element and you are guaranteed that k copies of it are present in the list. Furthermore, the element is equally likely to be in any position in the list. You design a randomized probe search algorithm, wherein each probe picks a position, uniformly at random, in the list and checks if the element that one is searching for is present there. Suppose your algorithm runs $\sqrt{n}$ number of probes. Derive, rigorously, an expression for the expected number of times it will find the element that is being searched for.

7. (2+1+2+5 points) Consider the following two decision problems :

3-Dimensional Matching (3DM) Given disjoint set X, Y , and Z , each of size n and given a set $T \subseteq X \times Y \times Z$ of ordered triples, does there exists a set of n triples in T so that each element of $X \cup Y \cup Z$ is contained in exactly one of these triples?

Exact Cover by 3-Sets (X3C) Given a finite set W with $|W| = 3q$ and a collection C of 3-element subsets of W , does C contain an exact cover for W , i.e., a subcollection $C' \subseteq C$ such that every element of W occurs in exactly one member of C' ?

- (a) Give an YES instance of **3DM** and a NO instance of **3DM**.
 - (b) Consider the **2DM** problem. Is it in P? Justify.
 - (c) Give an YES instance of **X3C** and a NO instance of **X3C**.
 - (d) Prove that **X3C** is NP-complete given the fact that **3DM** is a well known NP-Complete Problem.
8. (2+4+4 points) When finding cliques, it is natural to look for vertices of high degree. Suppose, however, that you want to find cliques consisting of vertices of relatively low degree. You will show that even this problem is NP-complete.

Low-Degree Clique (LDC): Given a graph $G = (V, E)$ and an integer k , does G have a clique of size at least k consisting entirely of vertices with degree is not greater than the median vertex degree of the entire graph.

By the *median vertex degree*, we mean the median value of the degree of all n vertices of the graph. The *median* of a set of n numbers is the $\lceil n/2 \rceil$ -smallest value of the set.

Show that the **LDC** problem is NP-complete by showing:

- (a) **LDC** is in NP.
- (b) Given a **CLIQUE** instance $G = (V, E)$, construct a **LDC** instance $G' = (V', E')$ such that you can reduce **CLIQUE** to **LDC**. Show that this construction time is polynomially (on the number of vertices of the original graph G) bounded.
- (c) Prove that you can solve **CLIQUE** iff you can solve **LDC**.